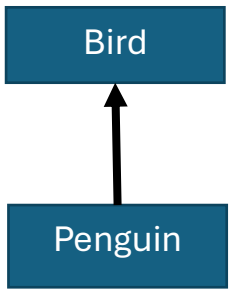
Module 10 Graded Programming Assignment – Inheritance and Polymorphism

In this assignment, we will demonstrate our understanding of inheritance, polymorphism, and interfaces by implementing a Bird hierarchy that includes a Penguin class to represent data from the Palmer Penguins dataset. We will create a base Bird class, derive a Penguin class, implement an interface, and override methods.

Our chief data scientist has asked us to think about the possibility of studying other birds.After doing some research using The Google™ we have determined that penguins are indeed birds, but they are a specialized type of bird known as a "flightless waterfowl":

https://www.berkeleybreathed.com/

Based on this information, we can use the strong "is-a" relationship between birds (a generalization) and penguins (a specialization) to create the following inheritance hierarchy:



Look at the instance variables we specified for our PenguinData class in Module 9; which of these attributes could moved to a generalized Bird class? I've highlighted them here:

```
// Private fields for encapsulation
private int sampleNumber;
private String species;
private double culmenLength; // renamed from "billLength"
private double culmenDepth; // renamed from "billDepth"
private double flipperLength;
```

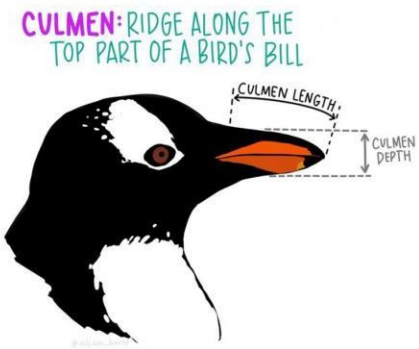
```
private double bodyMass;
private String sex;
```

This leaves flipperLength and species for our Penguin class (for this assignment we will rename our PenguinData class to Penguin to emphasize that it represents a virtual model of a real penguin via its attributes).

Note that flipperLength is actually a wing length that we could generalize on as well, but we'll keep that in the Penguin class and continue to refer to it as a flipper measurement. We could also generalize on a species of bird in the Bird class, but our application is referring specifically to the Palmer Penguin species so we will keep that in the Penguin class as well.

A culmen is actually a generic term for a bird's bill that is not specific to penguins so that can be moved to the Bird class, as can bodyMass, sex, and sampleNum. We renamed bill to culmen for this assignment so that we more accurately reflect the data's column names. Here's what those variables are measuring:

- What is a culmen?



Follow these steps to complete this assignment.

1. Fork and clone the public repository.

- The repository contains the CSV data and the following starter files:
  - Bird.java
  - Penguin.java
  - PenguinAnalyzer.java
  - PalmerPenguins.java
- Complete the author and submission date lines in the ID headers for the Bird.java and Penguin.java files that are not already completed.

2. Complete the design of the Bird base class

Bird.java represents a generic bird. Modify the class as follows:

- Complete the class declaration to specify that it implements the Comparable<Bird> interface
- Add the following private fields:

```
private int sampleNum;
private double culmenLength;
private double culmenDepth;
private double bodyMass;
private String sex;
```

- Implement an overloaded public Bird constructor that initializes all of the private fields using the "this" keyword. Here is the signature:

```
public Bird(int sampleNum, double culmenLength,
            double culmenDepth, double bodyMass, String sex) {
```

- Implement public getter methods for each private field.
- Implement a public compareTo method which compares a Bird "other" parameter. Include the @Override annotation before the header of the method. This method returns an int. The body for the method is included here:

```
int  result = Integer.compare(this.sampleNum, other.sampleNum);
if (result != 0) return result;

result = Double.compare(this.culmenLength, other.culmenLength);
if (result != 0) return result;

result = Double.compare(this.culmenDepth, other.culmenDepth);
if (result != 0) return result;

result = Double.compare(this.bodyMass, other.bodyMass);
if (result != 0) return result;

return this.sex.compareTo(other.sex);
```

3. Complete the design of the Penguin derived class

Penguin.java represents a Penguin. Modify the class as follows:

- Complete the class declaration to specify that it extends the Bird class.
- Add the following private fields:

```
private String species;
```

```
private double flipperLength;
```

- Implement an overloaded public Penguin constructor. Here is the signature:

```
public Penguin(int sampleNum, String species,
                double culmenLength, double culmenDepth,
                double bodyMass, String sex, double flipperLength) {
```

  This constructor must call "super" with the sampleNum, culmenLength, culmenDepth, bodyMass, and sex fields so the Bird class can initialize the fields that belong to it. It must then initialize the Penguin-specific fields (species and flipperLength) using the "this" keyword.

Expected Console Output (first and last 4 lines)

```
Testing compareTo method:
Comparison result between first two penguins: -1
Comparison result of p1 to itself (should be 0): 0
sampleNum=1, culmenLength=39.1, culmenDepth=18.7, bodyMass=3750.0, sex='MALE, species='Adelie', flipperLength=181.0


340          Gentoo     50.4         15.7         222.0        5750.0       MALE
341          Gentoo     45.2         14.8         212.0        5200.0       FEMALE
342          Gentoo     49.9         16.1         213.0        5400.0       MALE
Penguin data successfully written to PenguinOutput.txt.
```

Required Screenshots

- Take two screenshots of the console output – the top few lines and a separate screenshot of the bottom few lines.
- Run the "dir" command in the top-level project folder of a command prompt (or terminal if using a Mac) and take a screenshot to verify PenguinOutput.txt file has been correctly written.

Submission

- Use Git to submit your project files to your cloned copy of the repository, including all source files, the PenguinOutput.txt file, and the three required screenshots.
- Ensure your code follows proper indentation, statement length, comments, and meaningful variable names.
- Copy and paste your repository URL to the Canvas assignment as a comment.